

ATUALIZANDO VIEWS EM RESPOSTA A ALTERAÇÕES NA VIEW MODEL OU MODEL SUBJACENTE

Todas as classes de view model e model que são acessíveis a uma view devem implementar a interface **INotifyPropertyChanged**. A implementação desta interface em uma classe de view model ou model permite que a classe forneça notificações de alteração para quaisquer controles vinculados a dados (**data-bound**) na view quando os valores das propriedades subjacentes mudam.

Os aplicativos devem ser arquitetados para o uso correto da notificação de alteração de propriedade, atendendo aos seguintes requisitos:

- **Sempre disparar** um evento **PropertyChanged** se o valor de uma propriedade pública for alterado. Não presuma que o disparo do evento PropertyChanged possa ser ignorado devido ao conhecimento de como a vinculação XAML ocorre.
- **Sempre disparar** um evento PropertyChanged para quaisquer propriedades calculadas cujos valores sejam usados por outras propriedades na view model ou model.
- **Sempre disparar** o evento PropertyChanged ao final do método que faz a alteração da propriedade, ou quando o objeto for conhecido por estar em um estado seguro. O disparo do evento interrompe a operação ao envolver os manipuladores do evento de forma síncrona. Se isso acontecer no meio de uma operação, pode expor o objeto a funções de retorno (callback) quando ele estiver em um estado inseguro e parcialmente atualizado. Além disso, alterações em cascata podem ser acionadas por eventos

PropertyChanged. As alterações em cascata geralmente exigem que as atualizações estejam concluídas antes que a alteração em cascata seja segura para ser executada.

- **Nunca disparar** um evento PropertyChanged se a propriedade não mudar. Isso significa que você deve comparar os valores antigo e novo antes de disparar o evento PropertyChanged.
- **Nunca disparar** o evento PropertyChanged durante o construtor de uma view model se você estiver inicializando uma propriedade. Os controles vinculados a dados na view não estarão inscritos para receber notificações de alteração neste momento.
- **Nunca disparar** mais de um evento PropertyChanged com o mesmo argumento de nome de propriedade dentro de uma única invocação síncrona de um método de uma classe. Por exemplo, dada uma propriedade **NumberOfItems** cujo campo de suporte é o campo `_numberOfItems`, se um método incrementar `_numberOfItems` cem vezes durante a execução de um loop, ele deve apenas disparar a notificação de alteração de propriedade na propriedade **NumberOfItems** uma vez, após todo o trabalho estar concluído. Para métodos assíncronos, dispare o evento PropertyChanged para um determinado nome de propriedade em cada segmento síncrono de uma cadeia de continuação assíncrona.

Uma maneira simples de fornecer essa funcionalidade seria criar uma extensão da classe **BindableObject**. Neste exemplo, a classe **ExtendedBindableObject** fornece notificações de alteração, o que é mostrado no seguinte exemplo de código:

```

public abstract class ExtendedBindableObject : BindableObject
{
    public void RaisePropertyChanged<T>(Expression<Func<T>> property)
    {
        var name = GetMemberInfo(property).Name;
        OnPropertyChanged(name);
    }

    private MemberInfo GetMemberInfo(Expression expression)
    {
        // omitido por brevidade...
    }
}

```

A classe `BindableObject` do .NET MAUI implementa a interface `INotifyPropertyChanged` e fornece um método `OnPropertyChanged`. A classe `ExtendedBindableObject` fornece o método **`RaisePropertyChanged`** para invocar a notificação de alteração de propriedade e, ao fazer isso, usa a funcionalidade fornecida pela classe `BindableObject`.

As classes de view model podem então derivar da classe `ExtendedBindableObject`. Portanto, cada classe de view model usa o método `RaisePropertyChanged` na classe `ExtendedBindableObject` para fornecer notificação de alteração de propriedade. O exemplo de código a seguir mostra como o aplicativo multiplataforma `eShop` invoca a notificação de alteração de propriedade usando uma expressão lambda:

```
public bool IsLogin
{
    get => _isLogin;
    set
    {
        _isLogin = value;
        RaisePropertyChanged(() => IsLogin);
    }
}
```

O uso de uma expressão lambda dessa forma envolve um pequeno custo de desempenho porque a expressão lambda precisa ser avaliada para cada chamada. Embora o custo de desempenho seja pequeno e normalmente não cause impacto em um aplicativo, os custos podem se acumular quando houver muitas notificações de alteração. No entanto, o principal benefício dessa abordagem é que ela fornece segurança de tipo (**type safety**) em tempo de compilação e suporta a refatoração ao renomear propriedades.

FRAMEWORKS MVVM

O padrão **MVVM** está bem estabelecido no .NET, e a comunidade criou muitos frameworks que ajudam a facilitar esse desenvolvimento. Cada framework fornece um conjunto diferente de recursos, mas é padrão para eles fornecerem uma view model base com uma implementação da interface `INotifyPropertyChanged`. Recursos adicionais dos frameworks MVVM incluem comandos personalizados, auxiliares de navegação, componentes de injeção de dependência/localizador de serviços e integração de plataforma de UI. Embora não seja necessário usar esses frameworks, eles podem acelerar e padronizar seu desenvolvimento. O aplicativo multiplataforma `eShop` usa o **CommunityToolkit.Mvvm**. Ao escolher um framework,

você deve considerar as necessidades do seu aplicativo e os pontos fortes da sua equipe. A lista abaixo inclui alguns dos frameworks MVVM mais comuns para .NET MAUI:

- **.NET Community Toolkit MVVM**
- **ReactiveUI**
- **Prism Library**